

Final Test for SIC Coding and Programming Course

Date: 16/04/2026

Name: Xavier Gonzalez Arriola

Question 1.

Which of the following is the correct Python code to print 'Hi Python!' to the screen?

- 1 print(Hi Python!)
- 2 printf("Hi Python!")
- 3 **print('Hi Python!')** ←
- 4 D Print('Hi Python!')

Question 2.

What is the wrong way to use comments in Python programming?

- 1 Write comments after #(hash mark).
- 2 Write comments between """"and """".
- 3 Write comments between ""and "".
- 4 **Write comments after //.** ←

Question 3.

You want to get the name of the city a user live in from the user and store it in a variable called city. What is the correct code to do this?

- 1 city = input(Enter the name of the city you live in :)
- 2 **city = input('Enter the name of the city you live in :')** ←
- 3 input('Enter the name of the city you live in :) = city
- 4 city = scanf('Enter the name of the city you live in :')

Question 4.

If there is a list a having five elements as follows, what value does the pop (1) method return?

```
a = [10,20,30,40,50]
```

- 1 10
- 2 **20 ←**
- 3 30
- 4 40

Question 5.

If there is a list a [10, 20, 30, 40] having four integer elements, what value does index method return?

```
a = [10,20,30,40]  
a.index(40)
```

- 1 0
- 2 1
- 3 2
- 4 **3 ←**

Question 6.

Which of the following methods cannot be called by a list object?

- 1 pop
- 2 append
- 3 remove
- 4 **Get ←**

Question 7.

You want to create a list b with elements [1, 4, 9] using list a with three integers as elements [1, 2, 3]. In this case, the following list abbreviation code can be used. What is the appropriate code to fill in the blank?

```
a = [1, 2, 3]
b = [ _____ for x in a ]
```

- 1 x * 3
- 2 x ** 3
- 3 x ** 2 ←**
- 4 x * 2.

Question 8.

What is the output of the following Python code?

```
from collections import defaultdict

def countLetters(word):
    counter = defaultdict(int)
    for letter in word:
        if letter == 'c':
            counter[letter] += 2
            continue
        counter[letter] += 1
    return counter

a = countLetters('abcbaabca')
print(a)
```

- 1 {'a': 4, 'b': 3, 'c': 2}
- 2 {'a': 4, 'b': 2, 'c': 3}
- 3 {'a': 4, 'b': 3, 'c': 4} ←**
- 4 {'a': 3, 'b': 2, 'c': 2}.

Question 9.

What is the value of the counter variable in the following Python code?

```
def print_counter():  
    counter = 200  
  
counter = 100  
print_counter()  
print('counter=',counter)
```

- 1 300
- 2 200
- 3 150
- 4 100 ←

Question 10.

What is the function to check the data type of a when there is an object a grouping three integers as follows?

```
a = [10, 20, 30]
```

- 1 func()
- 2 id()
- 3 **type()** ←
- 4 list()

Question 11.

A stack is a last-in-first-out (LIFO) data structure in which the items can be added or removed from only one end. In Python, you can simply implement a stack as a list type. What is the output of the following code using the stack list?

```
stack = []  
for i in range(5):  
    stack.append(i)  
for j in range(2):  
    print(stack.pop(), end=" ")
```

```
1  0 1  
2  1 0  
3  3 4  
4  4 3 ←
```

Question 12.

A queue is a first-in-first-out (FIFO) data structure in which items added first are removed first. What is the output of the following code using a class in Python?

```
class Queue:

    def __init__(self):
        self.queue = []

    def is_empty(self):
        return True if len(self.queue) == 0 else False

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        return None if self.is_empty() else self.queue.pop(0)

    def size(self):
        return len(self.queue)

queue = Queue()
items = ["Apple", "Banana", "Tomato", "Strawberry", "Grapes"]
for item in items:
    queue.enqueue(item)
    if len(item) > 5:
        queue.dequeue()
print(queue.dequeue())
```

- 1 Banana
- 2 Tomato
- 3 Strawberry
- 4 **Grapes ←**

Question 13.

Binary search is a search method that reduces the search range of sorted lists by half. In the following code, which input value x performs the comparison operation the fewest times?

```
def bin_search(nums, x):  
    low, high = 0, len(nums) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if nums[mid] == x:  
            return mid  
        elif nums[mid] > x:  
            high = mid - 1  
        else:  
            low = mid + 1  
    return -1  
  
S = [11, 17, 26, 28, 37, 45, 53, 59, 77]  
x = int(input("Input the number to search: "))  
pos = bin_search(S, x)  
print(pos)
```

- 1 x = 11
- 2 x = 26
- 3 x = 37 ←**
- 4 x = 99

Question 14.

A hash table is a data structure that stores and extracts data by mapping keys to values by hashing.

What is the output of the following code when the hash table is implemented as follows?

```
class HashTable:

    def __init__(self, size):
        self.size = size
        self.table = {}
        for i in range(size):
            self.table[i] = []

    def hash(self, key):
        return key % self.size

    def get(self, key):
        return self.table[self.hash(key)]

    def put(self, key, value):
        bucket = self.table[self.hash(key)]
        if value not in bucket:
            bucket.append(value)

hashTable = HashTable(7)
hashTable.put(15, "Apple")
hashTable.put(20, "Banana")
hashTable.put(25, "Tomato")
hashTable.put(30, "Strawberry")
```

- 1 **['Apple'] ←**
- 2 ['Banana']
- 3 ['Tomato']
- 4 ['Strawberry']

Question 15.

Bubble sort is a sorting algorithm that compares two adjacent elements and swaps places if the order is reversed. In the following bubble sort algorithm, if the input list S is given as [20, 30, 40, 50, 10], how many swap operations should all be performed?

```
def bubblesort(S):  
    n = len(S)  
    count = 0  
    for i in range(n):  
        for j in range(n - 1):  
            if S[j] > S[j + 1]:  
                S[j], S[j + 1] = S[j + 1], S[j]  
                count += 1  
    return count  
  
S = [20, 30, 40, 50, 10]  
count = bubblesort(S)  
print(count)
```

- 1 3
- 2 4 ←**
- 3 5
- 4 6

Question 16.

Merge sort is a sorting algorithm that splits an unsorted list into two, sorts them, and then merges the two sorted lists. If you are trying to sort an already sorted list with N elements by merge sort, what is the closest to the time complexity of the merge sort algorithm according to N given?

- 1 $O(\log N)$
- 2 $O(N)$
- 3 $O(N \log N)$ ←**
- 4 $O(N^2)$

Question 17.

Quicksort is a sorting algorithm that divides and conquers into two partial lists based on a given pivot. The partition1() function uses a strategy to pivot the first element of the list. In this case, what is the appropriate output of the following code?

```
def partition1(S, low, high):  
    pivot, left, right = S[low], low, high  
    while left < right:  
        while left < high and S[left] <= pivot:  
            left += 1  
        while right > low and pivot <= S[right]:  
            right -= 1  
        if left < right:  
            S[left], S[right] = S[right], S[left]  
    S[low], S[right] = S[right], S[low]  
    return right  
  
S = [5, 2, 7, 3, 4, 6]  
partition1(S, 0, len(S) - 1)  
print(S)
```

- 1 **[3, 2, 4, 5, 7, 6] ←**
- 2 [2, 3, 4, 5, 6, 7]
- 3 [4, 3, 2, 5, 6, 7]
- 4 [3, 4, 2, 5, 7, 6]

Question 18

Which of the following is grouped only as an algorithm with a time complexity of $O(N^2)$ in the worst case?

- 1 Insertion sort, Merge sort
- 2 Merge sort, Binary search
- 3 Binary search, Selection sort
- 4 **Selection sort, Quick sort ←**

Question 19.

Divide-and-conquer is an algorithm design strategy that divides and conquers the input case of a given problem into two or more small input cases. If one coin out of 100 coins of the same size has different weights, you can use a balanced scale to find coins of different weights using a divide-and-conquer strategy. At this time, in the worst case, how many comparisons must be made to find coins with different weights?

- 1 4
- 2 6 ←
- 3 8
- 4 10

Question 20.

The fib1(), fib2(), and fib4() functions given below are functions that implement the algorithm to find the term of the Nth Fibonacci sequence, respectively. For these functions, we try to calculate repeatedly (e.g., more than 10,000 times) the problem of obtaining terms in the Nth Fibonacci sequence for any number N that is very large (e.g., greater than 10,000). What is the most efficient algorithm in terms of time complexity in this case?

```
def fib1(n):  
    if n == 0 or n == 1:  
        return n  
    else:  
        return fib1(n - 1) + fib1(n - 2)
```

F = {0: 0, 1: 1}

```
def fib2(n):  
    if n in F:  
        return F[n]  
    else:  
        F[n] = fib2(n - 1) + fib2(n - 2)  
        return F[n]
```

```
def fib4(n):  
    if n <= 1:  
        return n  
    else:  
        a, b = 0, 1  
        for i in range(2, n + 1):  
            a, b = b, a + b  
        return b
```

- 1 fib1()
- 2 fib2()
- 3 **fib4()** ←
- 4 They are all the same.